

北京理工大学第十七届“连山科技”程序设计大赛

The 17-th BIT Campus Programming Contest

Sponsored by

连山科技
LSSEC TECH

网络选拔赛
Online Round



题目列表
Problem List

A	赛前须知
B	turn
C	Warp Shuffle
D	塔学疑云
E	string
F	出题出题人
G	分辨矩阵

请勿在比赛开始前翻阅试题！

Do not open before the contest has started.

2023 年 10 月 21 日

Problem A. 赛前须知

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

请注意，这是一道编程题。

亲爱的同学们，

欢迎你们出席北京理工大学第十七届“连山科技”程序设计大赛（网络选拔赛）。我们十分高兴地迎接你们的到来。在此，我谨代表竞赛组委会，向各位同学们，表示热烈的欢迎和衷心的感谢。

下面，我将为大家再次强调本次比赛的竞赛规则，请认真阅读。

1. 本次比赛是线上赛。队伍中的每位选手可以使用一台计算机登陆考试系统作答。
2. 本次比赛采用 ACM 赛制。ACM 赛制以各个队伍的数量为第一关键字进行排序，当过题数量相同时，以罚时作为第二关键字进行排序，罚时的计算方式是：已通过的题目的 AC 的时间的总和 + 这些题目提交前错误的次数（不含编译错误）*20 分钟。
3. 本次比赛允许的竞赛语言包括：C, C++, Java, Python。
4. 本次比赛**允许**使用互联网，您可以使用**截止比赛开始前**互联网上公开的任何资料。但是，不允许队伍之间相互交流。赛后将通过**代码查重**等方式筛选可能违反比赛规则的队伍，并视情况给予相应的处理。
5. 本次比赛的题目**不按难度顺序排序**，当遇到不会做的题目时，请继续阅读下一道题目。
6. 如有疑问，可通过比赛系统向裁判组提出。

为了确保各位同学都已经阅读并理解上述内容，请大家提交一份回执。即编写一个程序，输出以下内容：

```
I have read the rules of the competition above and fully understand the meaning of each rule. I understand and acknowledge the following:  
1. Each player on my team can use one computer for the competition, and multiple computers can be used simultaneously.  
2. During the competition, I can use the Internet but cannot communicate with other teams.  
3. The problems in this competition are not ordered by difficulty, so if I encounter a problem I don't know how to solve, I should continue reading the next problem.
```

Input

本题不需要读取输入数据。

Output

按照题目要求，输出你提交的回执。

你的输出应当恰好包含题目中所示的**四行**内容，请不要擅自添加额外的换行符。

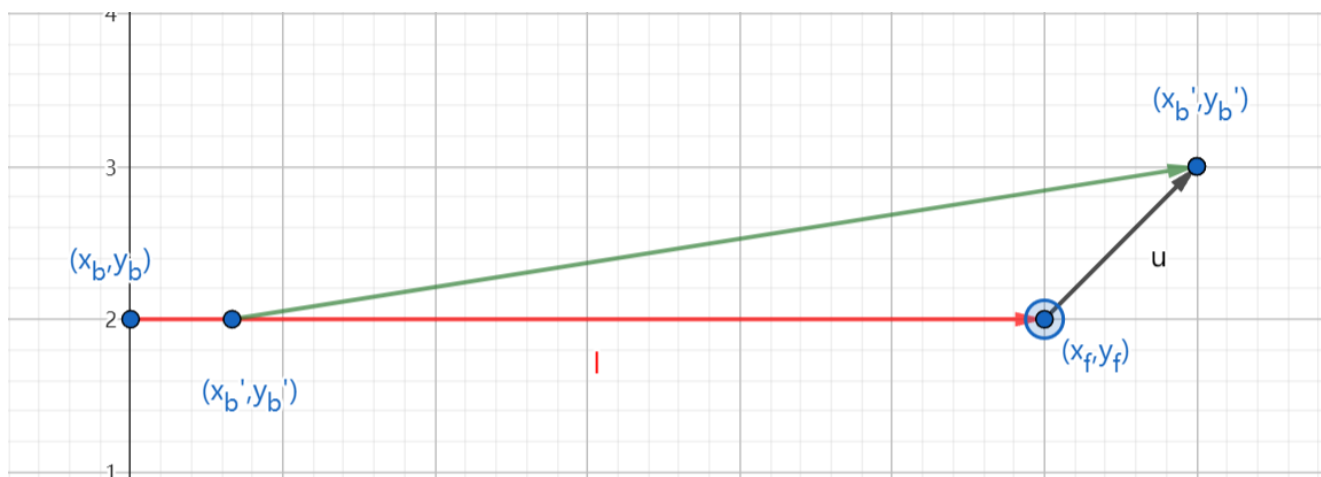
Note

由于竞赛平台的限制，输入数据其实不是空的。

Problem B. turn

Input file: standard input
 Output file: standard output
 Time limit: 1 second
 Memory limit: 256 megabytes

zzl 在一个宽度为 d 的狭长道路上靠着最右边骑着自行车，突然前面有一堆 ljh 冲了过来，zzl 想要掉头，zzl 可以进行若干次前进和后退来完成掉头，由于倒车很慢，zzl 想要用**最少的倒车次数**进行掉头，请问 zzl 至少要倒车几次。



在任意短 Δt 时间内自行车的运动方式

如上图所示，道路可以抽象成一个由两条无限长平行直线构成的区域。zzl 驾驶的自行车可以抽象为一个在二维平面上长为 l 的线段，自行车的车轮在线段的最前和最后端。上图展示了自行车以 45 度进行转向的示意图，红色和绿色分别表示 Δt 时间前后的自行车位置。自行车进行转弯时候的具体定义如下：

自行车的转弯通过前轮的方向控制，假设在某一时刻前后轮的位置分别为： $(x_f, y_f), (x_b, y_b)$ ，前轮朝向方向 $\mathbf{u}$ 。则在 Δt 时间后，前轮将会沿着方向 $\mathbf{u}$ 移动一段距离，即新位置 (x'_f, y'_f) 满足 $(x'_f, y'_f) = (x_f, y_f) + \Delta t \times \mathbf{u}$ ；后轮会沿着原来的自行车方向移动一段距离，并且保持自行车形状不变，即新位置 (x'_b, y'_b) 满足向量 $(x'_b - x_b, y'_b - y_b)$ 和向量 $(x_f - x_b, y_f - y_b)$ 的方向相同，且 $(x'_f - x'_b)^2 + (y'_f - y'_b)^2 = l^2$ 。

由于自行车是后轮驱动的，所以转弯的方向（向量 $\mathbf{u}$ 的方向）不能和车身方向（向量 $(x_f - x_b, y_f - y_b)$ 的方向）垂直。我们可以认为转弯的方向与车身方向夹角大于 90 度就是在倒车。如果在连续的两个 Δt 时间内夹角发生了改变，且第二个夹角大于 90 度，则认为发生了一次倒车。

我们的调头指的是车身方向最终和最初相比转变了 180 度。

Input

一行，两个正整数， $d, l (1 \leq d, l \leq 1000)$ 表示路的宽度和自行车长度。

Output

一行，如果能成功掉头，输出一个整数，表示最小倒车次数。否则输出“gg”（不包含引号）表示无法成功掉头。

Example

standard input	standard output
10 10	gg

Note

对于样例，可以证明，只有路宽度严格大于 10 才能成功掉头。

Problem C. Warp Shuffle

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

不会吧，不会吧，都 AI 大模型时代了，还有人不会写 CUDA？

多处理器以一组 32 个并行线程（称为 warp）为单位创建、管理、调度和执行线程。组成一个线程的各个线程从相同的程序地址开始，但它们有自己的指令地址计数器和寄存器状态，因此可以自由分支和独立执行。（CUDA C++ Programming Guide Release 12.2）

函数定义如下：

- `T __shfl_up_sync(unsigned mask, T var, unsigned int delta, int width=warpSize);`
- `T __shfl_down_sync(unsigned mask, T var, unsigned int delta, int width=warpSize);`
- `T __shfl_xor_sync(unsigned mask, T var, unsigned int delta, int width=warpSize);`

在本题中，我们可以简单将 warp 理解为一个长为 32 的数组 $a[0], \dots, a[31]$ ，这些指令可以快速地对指定的下标集合进行「数据传输」。

mask 用于确定参与数据传输的下标集合 o 。设 $f(x)$ 为 x 二进制表示中所有为 1 的位的集合，如果 $f(\text{mask}) \subseteq f(p)$ ，那么下标 p 将参与此次的数据传输。

例如若 $\text{mask} = (10)_{10} = (01010)_2$ ，则 $f(10) = \{1, 3\}$ 。 $(8)_{10} = (01000)_2$ ， $f(8) = \{3\}$ ，则 8 不符合条件；而 $(31)_{10} = (11111)_2$ ， $f(31) = \{0, 1, 2, 3, 4\}$ ，则 31 符合条件。

delta 和函数名共同确定传输的所有「下标对」。例如对于 `__shfl_up_sync` 来说，对于任意下标 $x \in o$ ，若存在下标 y 满足 $y \in o$ 且 $y = x - \text{delta}$ ，则将 $a[x]$ 传输到 $a[y]$ ，记为 $x \rightarrow y$ 。若不存在这样的 y ，则无事发生。

注意：每次数据传输对于所有下标来说是同时发生的，没有先后次序。因此若存在 $z \rightarrow y, y \rightarrow x$ ，则 $a[y]$ 接收到的是数据传输前的 $a[z]$ ，同理 $a[x]$ 接收到的是数据传输前的 $a[y]$ 。

在本题中，「数据传输」有以下三种方式：

- 0: `__shfl_up_add(mask, delta)`：从相对于调用者下标较低的下标复制并累加，即 $a[p] += a[p - \text{delta}]$ （若 $p \in o, p - \text{delta} \in o$ ）
- 1: `__shfl_down_add(mask, delta)`：从相对于调用者下标较高的下标复制并累加，即 $a[p] += a[p + \text{delta}]$ （若 $p \in o, p + \text{delta} \in o$ ）
- 2: `__shfl_xor_add(mask, delta)`：从调用者下标的异或的下标复制并累加，即 $a[p] += a[p \oplus \text{delta}]$ （若 $p \in o, p \oplus \text{delta} \in o$ ，其中 $\oplus$ 为异或）

你的任务就是模拟这些指令，得到最终的 warp。

提示：所谓异或，即按二进制逐位异或。异或的规则为： $1 \oplus 1 = 0, 0 \oplus 0 = 0, 1 \oplus 0 = 1, 0 \oplus 1 = 1$ ，例如 $(001)_2 \oplus (101)_2 = (100)_2$ ，即 $(1)_{10} \oplus (5)_{10} = (4)_{10}$ 。

Input

第一行一个正整数 $T(1 \leq T \leq 1000)$ ，表示数据组数。

接下来 T 组数据，对于每组数据：

第一行一个正整数 $n(1 \leq n \leq 10)$ ，表示操作数。

第二行 32 个整数 $a_0, a_1, \dots, a_{31}(0 \leq a_i \leq 31)$ ，表示 warp 的初值。

接下来 n 行，每行 3 个整数 $op(0 \leq op \leq 2), mask(0 \leq mask \leq 31), delta(0 \leq delta \leq 31)$ ， op 表示该操作是哪个函数， $mask$ 和 $delta$ 的含义如上文所示。

Output

一共 T 行，每行一个整数，表示所有操作结束后， $a_0 \oplus a_1 \oplus \dots \oplus a_{31}$ 的值， $\oplus$ 为按位异或 (BITWISE-XOR)，即整个 warp 的异或和。

Example

standard input																													
1																													
5																													
0 3 19 21 0 29 0 30 24 25 0 6 5 1 15 8 6 3 17 4 31 14 10 27 25 25 6 22 0 29 18 31																													
0 22 0																													
1 25 5																													
1 29 23																													
2 1 29																													
2 1 12																													
standard output																													
38																													

Note

对于给出的样例：

第 1 次操作为 `__shfl_up_add(22,0)`。 $mask$ 的二进制表示为 $(22)_{10} = (10110)_2$ ，符合条件的下标为： $(22)_{10} = (10110)_2, (23)_{10} = (10111)_2, (30)_{10} = (11110)_2, (31)_{10} = (11111)_2$ ，因此数据传输情况如下：

- $22 \leftarrow 22$
- $23 \leftarrow 23$
- $30 \leftarrow 30$
- $31 \leftarrow 31$

第 2 次操作为 `__shfl_down_add(25,5)`。 $mask$ 的二进制表示为 $(25)_{10} = (11001)_2$ ，符合条件的下标为： $(25)_{10} = (11001)_2, (27)_{10} = (11011)_2, (29)_{10} = (11101)_2, (31)_{10} = (11111)_2$ ，但无同时满足条件的下标对。

第 3 次操作为 `__shfl_down_add(29,23)`。`mask` 的二进制表示为 $(29)_{10} = (11101)_2$ ，符合条件的下标为： $(29)_{10} = (11101)_2, (31)_{10} = (11111)_2$ ，但无同时满足条件的下标对。

第 4,5 次操作以此类推。

最终的 `warp` 为：0, 4, 19, 29, 0, 54, 0, 36, 24, 54, 0, 36, 5, 4, 15, 29, 6, 32, 17, 66, 31, 39, 20, 76, 25, 39, 6, 76, 0, 32, 36, 66，异或和为 38。

Problem D. 塔学疑云

Input file: standard input
Output file: standard output
Time limit: 2 seconds
Memory limit: 256 megabytes

小 C 最近爱玩杀戮尖塔，尤其沉迷于玩机宝 (故障机器人)。现在他模拟了一局对战。

小 C 的牌组里有以下几种卡牌：

- 漆黑 +：生成一个黑暗充能球。使用所有黑暗充能球的被动能力。
- 扩容 + K ：获得 K 个空的充能球栏位 (假设栏位数量无上限)。
- 碎片整理 + M ：获得 M 点集中。
- 偏差认知 + (A, B) ：获得 A 点集中。在每回合开始时，失去 B 点集中。
- 递归 +：激发 (并移除) 最右边的充能球，然后生成一个刚才被激发的充能球 (即生成时的计数值等于刚被激发的球的计数值)。如果没有充能球，则不操作。

为了防止你看不懂卡牌，小 C 给你解释了一些关键名词：

- 充能球：初始有 3 个空栏位。生成充能球时，将它填入最右边的空充能球栏位，如果充能球栏位已满，会先激发 (并移除) 最右边的充能球，并将所有充能球右移一个栏位，再在空的栏位生成应该生成的充能球。回合结束时，使用所有充能球的被动能力一次。
- 黑暗充能球：充能球的一种。被动：计数 $+= (\max(6 + \text{集中}, 0))$ 。激发：对生命值最低的敌人 (注：本题中实际上只有一名敌人) 造成等同于计数值的伤害。计数值在刚生成时为 6。
- 集中：人物属性，初始为 0。会影响黑暗充能球的被动能力。集中可以是负数。
- debuff：偏差认知会产生一个 debuff：“回合开始时，失去 B 点集中”。debuff 效果可叠加。



生成充能球的示意图，最右边的球被激发，并生成到最右边的第一个空位。

同时，小 C 也可以选择结束回合。回合结束后，敌人完成行动后，我方下一回合开始。

这里，敌人的血量为 10^{100} ，行动只有“造成 0 点伤害”。

小 C 嫌杀戮尖塔动画太慢了，于是想请你算一算，完成所有操作后，总共能造成多少伤害。

由于答案可能很大，请您输出答案除以 $10^9 + 7$ 的余数。

注意到， $(a + b) \% p = (a \% p + b \% p) \% p$ ，其中 $\%$ 是取模操作，所以您可以在计算的过程中取模。

Input

输入一个整数 q ，表示操作次数。

接下来 q 行，每行表示一个操作：

- 1 表示打出“漆黑 +”。
- 2 K 表示打出“扩容 +K”
- 3 M 表示打出“碎片整理 +M”
- 4 A B 表示打出“偏差认知 +(A,B)”
- 5 表示打出“递归 +”
- 6 表示结束当前回合。

其中：

$$1 \leq K, M, A, B \leq 10^9$$

$$1 \leq q \leq 10^6$$

Output

输出一个整数，表示最后的伤害值，由于结果可能过大，您需要输出结果除以 $10^9 + 7$ 的余数。

Example

standard input	standard output
14 1 1 1 1 2 1 1 5 3 2 1 5 4 4 1 6 5 5	186

Note

对于给出的样例，完成每次操作后的充能球计数值如下：

time 1: [12]

time 2: [12 18]

time 3: [12 18 24]

time 4: [12 18 24]

time 5: [12 18 24]

time 6: [12 18 24 30]

time 7: [30 12 18 24]

time 8: [30 12 18 24]

time 9: [14 38 20 26]

time 10: [26 14 38 20]

time 11: [26 14 38 20]

time 12: [38 26 50 32]

time 13: [32 38 26 50]

time 14: [50 32 38 26]

Problem E. string

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

Today, DLee is preparing for ‘17th-bitcpc-online’, and he is assigned to set a problem about string.
He thinks a while a write down this description.

DLee has a string S of length n , which is composed of visible characters (that means each character has ACSII between 33 and 126, without blank space). Now he will do m operations in order, the i -th operation can be described as (x_i, y_i) , where x_i and y_i are two different visible characters. DLee will ‘swap’ all these two kinds of characters and get a new string, and then continue with the next operation.

A ‘swap’ of characters x and y for string S means that replace all characters x with y and replace all characters y in S with x **at the same time**.

For example, the initial string S is ‘abcd’(without quotation marks), the first operation is (a, c) , and the second operation is (z, d) . The string S will first become to ‘cbad’ after the first operation, and then become to ‘cbaz’ after the second operation.

Now DLee want to know the final string after these n operations.

Input

The first line contains two integers $n, m(1 \leq n, m \leq 10^5)$ — the length of string S and the number of operations.

The second line contains only one string S with length n .

The i -th of the next m lines contains two visible characters x_i, y_i — the i -th operation is (x_i, y_i) .

Output

Print a single line with the final string after m operations.

Examples

standard input	standard output
4 2 abcd a c d z	cbaz
5 3 @#\$\$% # ! @ ! ? !	?@\$%?

Note

Each visible character can be store as a 'char'.

In the second example, the string will be changed like '@#\$\$@' → '@!\$%@' → '!@\$%!' → '?@\$%?'

Problem F. 出题出题人

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

你是百丽宫校赛的出题人，你正在忙于给校赛的网选出题，当你在出题平台 polygon 上输入了长长的一段题面故事后，你点击 save 按钮，却发现网站跳转进入了登录页面——你写题面的时间过长，登录失效了！

你相当沮丧，因为这意味着你需要花同样多的时间再次输入之前输入过的题面，并且在途中还要花更多的时间点击保存按钮。你知道登录失效的时间是由平台内部决定的定值 l ，当你在输入完题面点击“保存”按钮时，若距离你上一次保存的时间超过了保存阈值 l ，则你会被强制跳转到登录界面，且你在上一次保存之后输入的所有内容均会丢失。

理论最保险的方式是频繁点击“保存”按钮，但你并不想这么做，因为每次点击“保存”按钮，平台会有一段将数据上传并刷新网页的时间 t_{flush} ，在此期间你无法做任何操作，这段时间显得相当难熬。幸运的是，你已经知道了网站内部的参数 l 在两个已知常数 x 和 y 之间，即满足 $x \leq l \leq y$ 。由于你并不知道网站选择参数的方式，因此你将该参数视为在 $x \sim y$ 之间的 $y - x + 1$ 个整数之内均匀随机挑选一个。

现在，你有一篇 n 个字的题面需要输入，每个字你需要 1 单位的时间输入，输入完任何一个字后你均可以点击保存按钮，每次点击保存按钮都将额外花费 t 单位的时间。最开始，上一次保存的时间 t_{last} 被视为 0，随后，当你在 i 时刻点击保存按钮时，网站会立即检查 $i - t_{last}$ 与 l 的大小关系，若 $i - t_{last} \leq l$ 则保存成功，否则保存失败；若保存成功，则第 $i + t_{flush}$ 时刻你输入的内容会被保存，否则，第 $i + t_{flush}$ 时刻你的页面会立即重置为你上一次保存的状态，即你在中间过程中输入的额外内容被清除。无论保存成功还是失败，你都在 $i + t_{flush}$ 时刻才能得知结果，并且上一次保存时间也会被直接置为 $i + t_{flush}$ 。你希望知道，当你采用最优策略（即尽可能使期望时间最小的策略）时，你输入整篇题面并全部保存下来的最小期望时间。

例如，你有一篇长度为 5 的文章，阈值 $l = 2$ ，等待时间 $t_{flush} = 1$ ，你先输入了 3 个词，并点击保存按钮，你会在 4 时刻得到保存失败的信息，此时你的已输入词数会被重置为 0，你输入 2 个词，并再次点击保存按钮，点击时为 6 时刻，你会在 7 时刻得到保存成功的信息，此时上一次保存的时刻也被设置为 7，你可以继续进行内容的编辑。

Input

一行四个正整数 n, x, y, t_{flush} ($1 \leq n, x, y, t_{flush} \leq 100, x \leq y$)，分别代表题面长度，保存阈值的范围，按下保存按钮产生的额外时间消耗。

Output

输出一行一个整数 ans 代表你求得的最小期望时间对 $10^9 + 7$ 取模后的答案。具体的，若你求得的答案的期望值为分数 $\frac{u}{v}$ ，则输出的答案需要满足 $0 \leq ans < 10^9 + 7$ ，且 $ans \times v \equiv u \pmod{10^9 + 7}$ ，可以证明这样的 ans 在 $0 \sim 10^9 + 6$ 间只有一个。

Examples

standard input	standard output
5 1 5 1	9
21 1 4 10	500000149
21 1 5 10	600000137
89 23 97 11	413333463

Note

对于样例 2，我们采用的策略为，先尝试输入 3 个词后保存，若成功则一直采用每 3 个词保存一次的方式，若失败则再尝试 2 个词保存是否成功，此后即可得到真实的阈值，并采用最优策略。

具体的：

- 若阈值为 1，则前两次尝试失败浪费了 25 单位时间，此后消耗 $21 \times 11 = 231$ 单位时间，共 256；
- 若阈值为 2，则第一次尝试失败浪费了 13 单位时间，此后消耗 $21 + 10 \times 11 = 131$ 单位时间，共 144；
- 若阈值为 3 或 4，则共花费 $21 + 10 \times 7 = 91$ 单位时间；

期望值为 $\frac{1}{4} \times (256 + 144) + \frac{2}{4} \times 91 = 145.5 = \frac{291}{2}$ ，我们发现 $500000149 \times 2 = 1000000298 = (10^9 + 7) + 291 \equiv 291 \pmod{10^9 + 7}$ ，故答案为 500000149。

可以证明，没有期望值更优的其他策略。

样例 1、3、4 的实际期望值分别为 $9 \frac{664}{5} \frac{9508}{75}$ 。

Problem G. 分辨矩阵

Input file: standard input
Output file: standard output
Time limit: 1 second
Memory limit: 256 megabytes

有若干大小为 $n \times m$ 的 01 矩阵，它们互不相同，该如何区分呢？

聪明的你想到，其实不需要所有位置逐一比较，只需要选取一些特殊的位置就可以区分，这样大大减少了工作量。

举个栗子，下面有四个 2×3 的 01 矩阵。我们只需要根据 (0,0) 和 (0,1) 这两个位置即可将它们全部区分。

$$\begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

于是问题来了，最少需要选取几个位置呢？相信聪明的你一定能解决。

Input

第一行一个正整数 $T(1 \leq T \leq 50)$ ，表示数据组数。

接下来 T 组数据，每组数据格式如下：

第一行三个整数 $n, m, k(1 \leq n, m \leq 10, 2 \leq k \leq 6)$ ，表示 01 矩阵的大小，以及个数。

接下来 k 个矩阵，每个矩阵 n 行，每行一个长为 m 的 01 串。

Output

输出 T 行，每行一个正整数，表示答案。

Example

standard input	standard output
1 3 6 3 111111 111111 110111 111110 110111 111111 111111 111111 111111	2

Note

对于给出的样例，三个矩阵分别为：

$$\begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

取 $(2, 2), (1, 2)$ 即可分辨这三个矩阵，因此答案为 2。